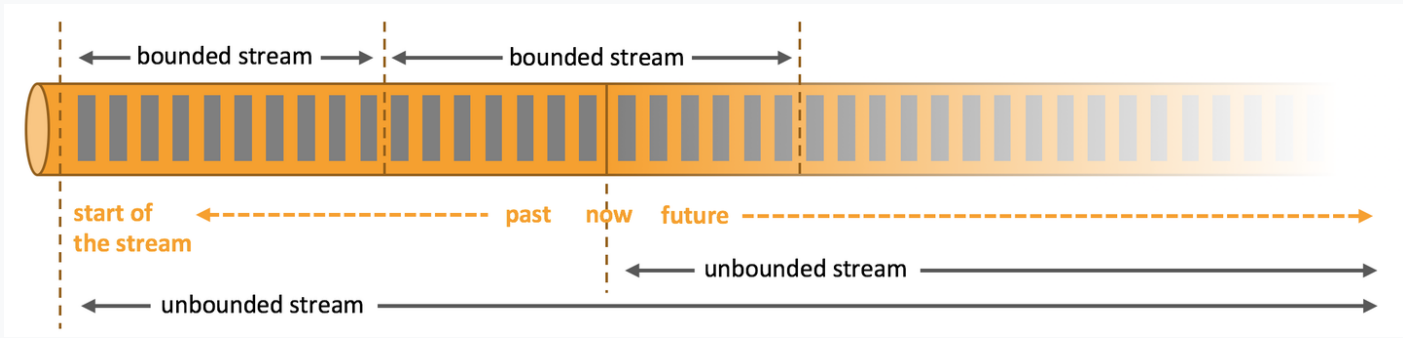
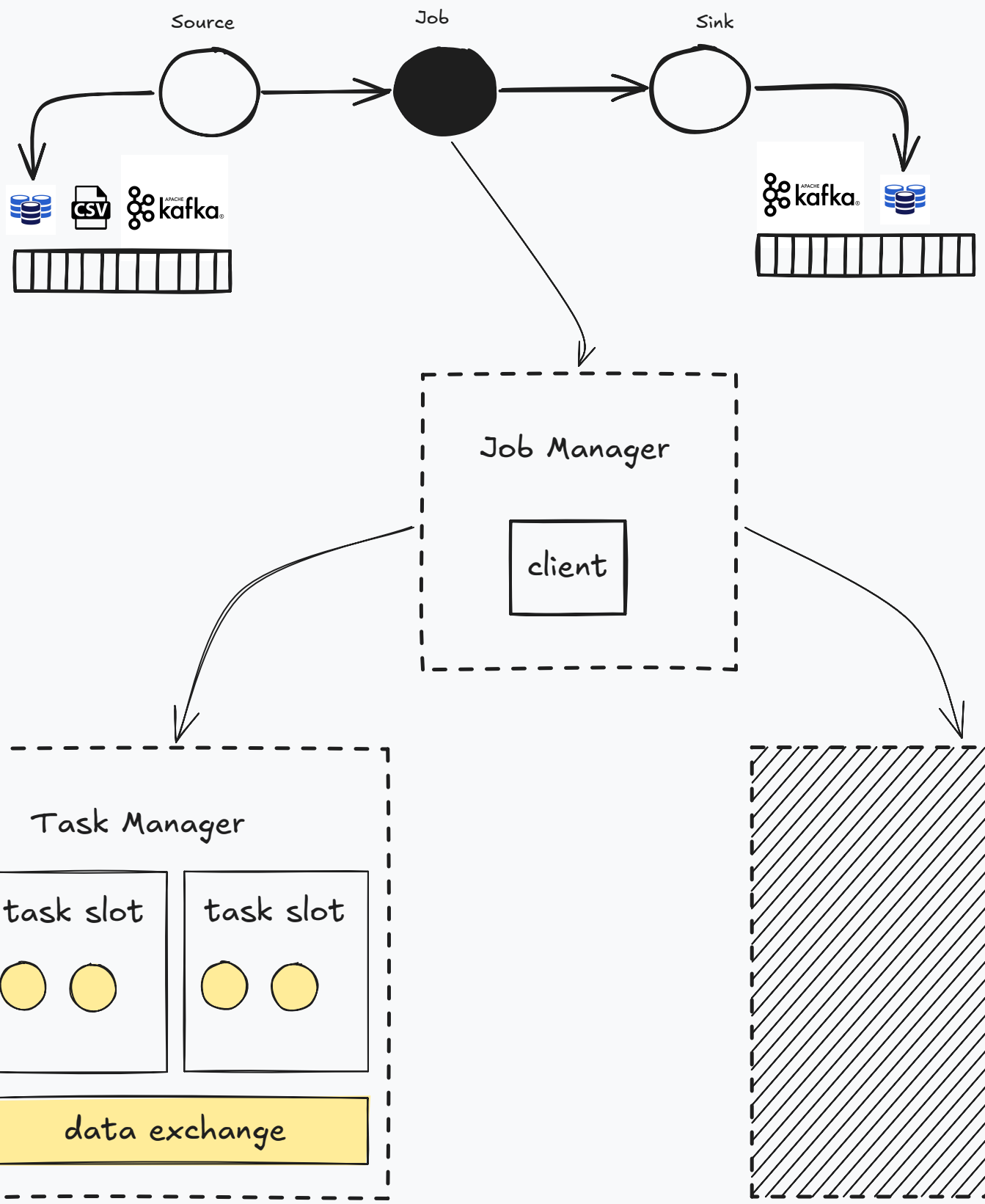
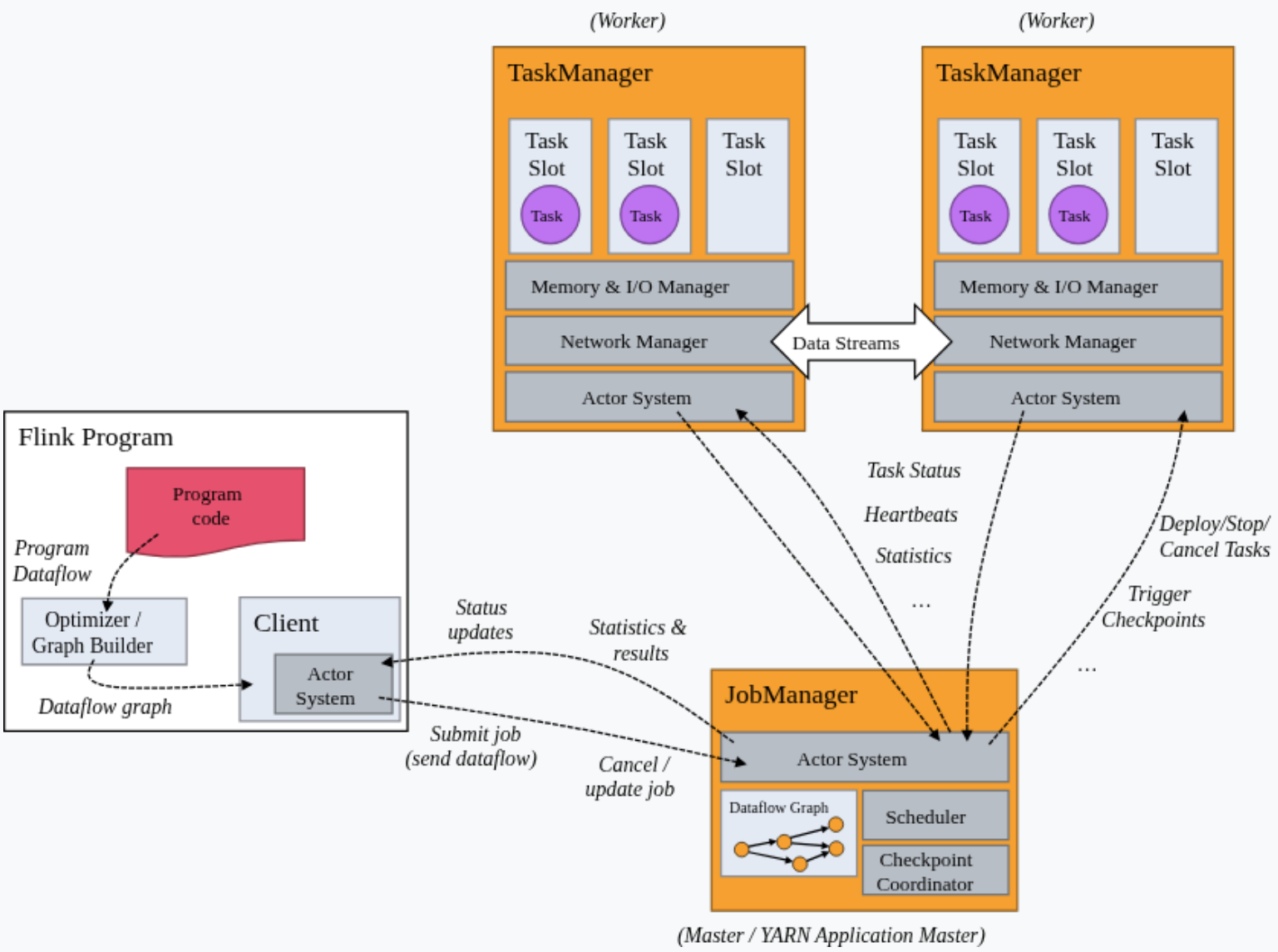
Anatomy of a Apache Flink





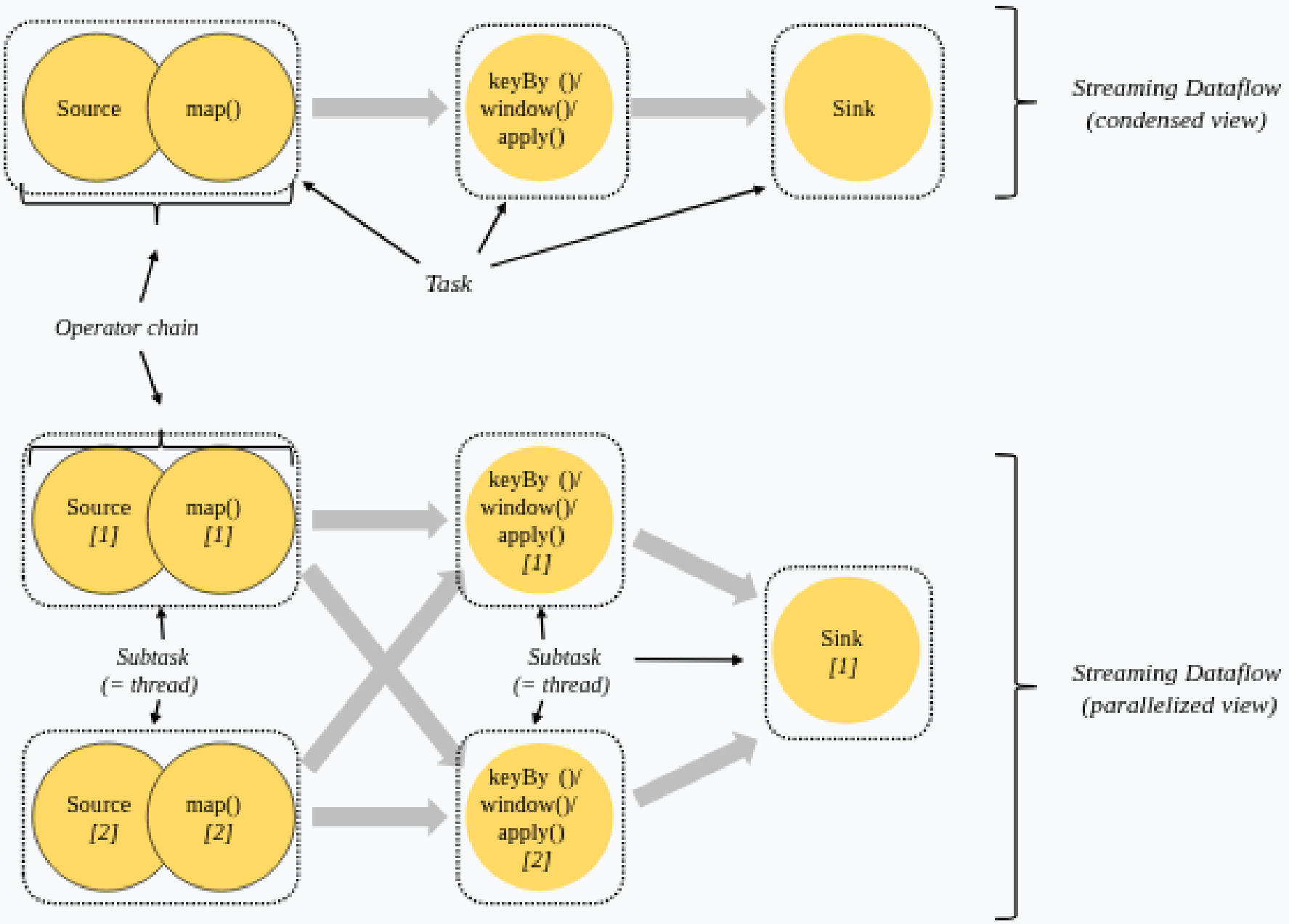
The TaskManagers (also called workers) execute the tasks
The number of task slots indicates how many tasks can be processed concurrently.

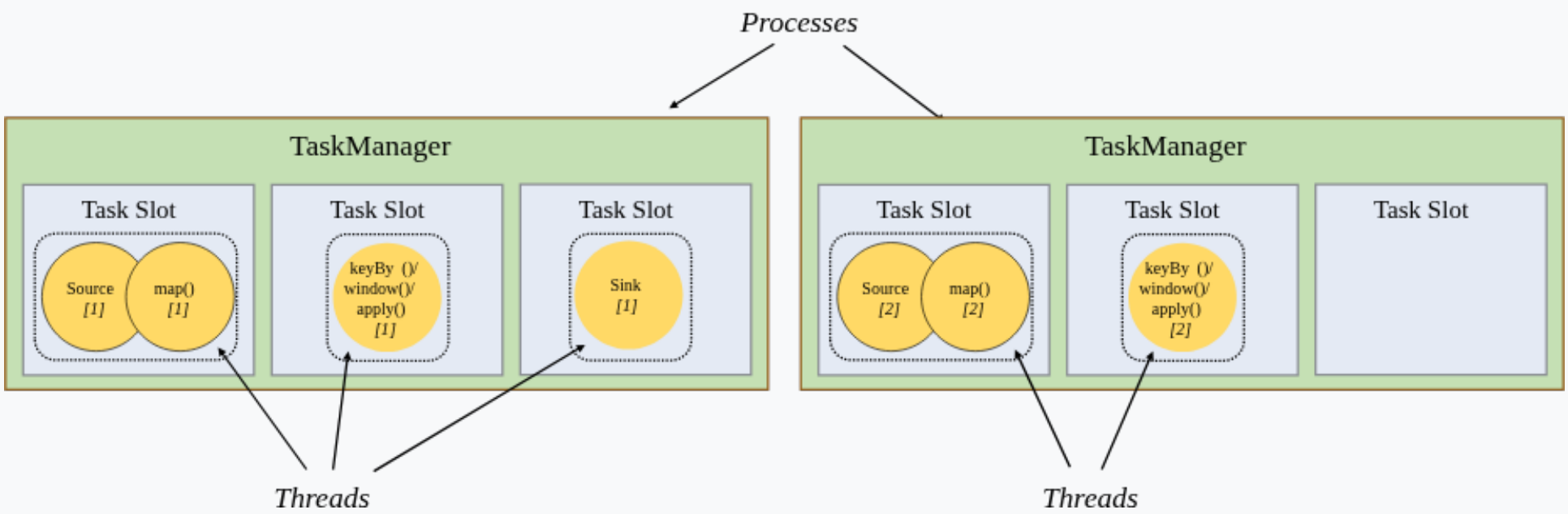
JobManager / TaskManager



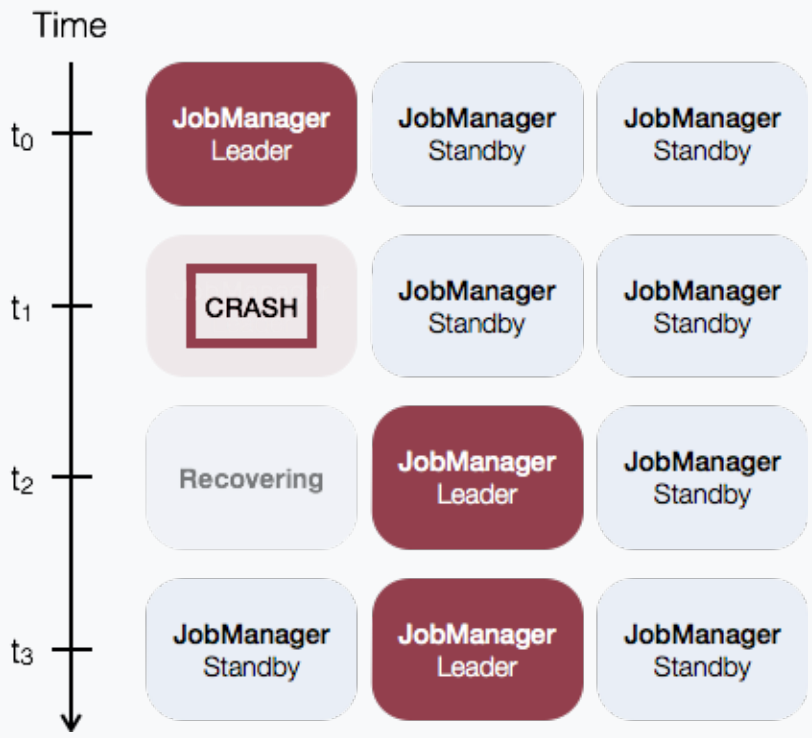
Flink operators

Flink chains operator subtasks together into tasks.
Each task is executed by one thread.





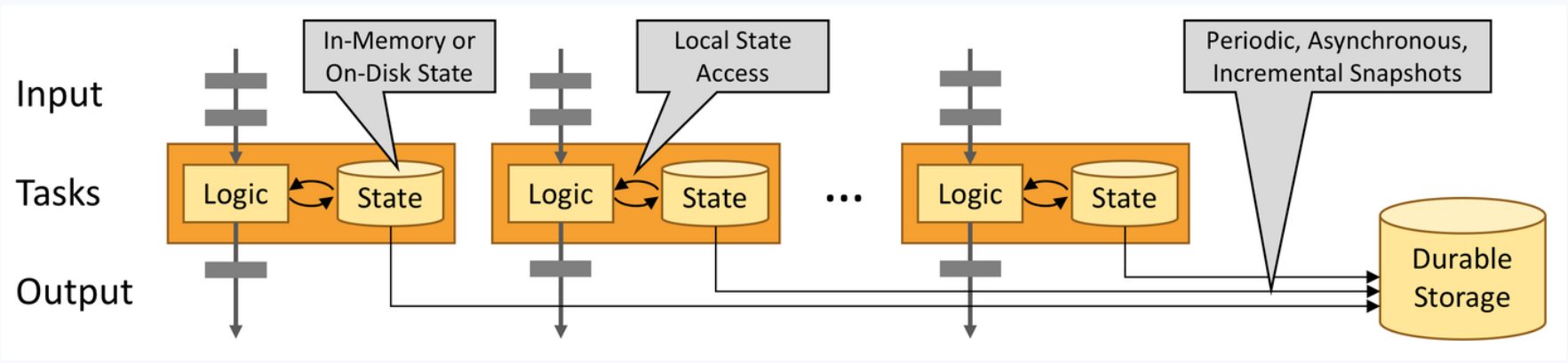
High Availability



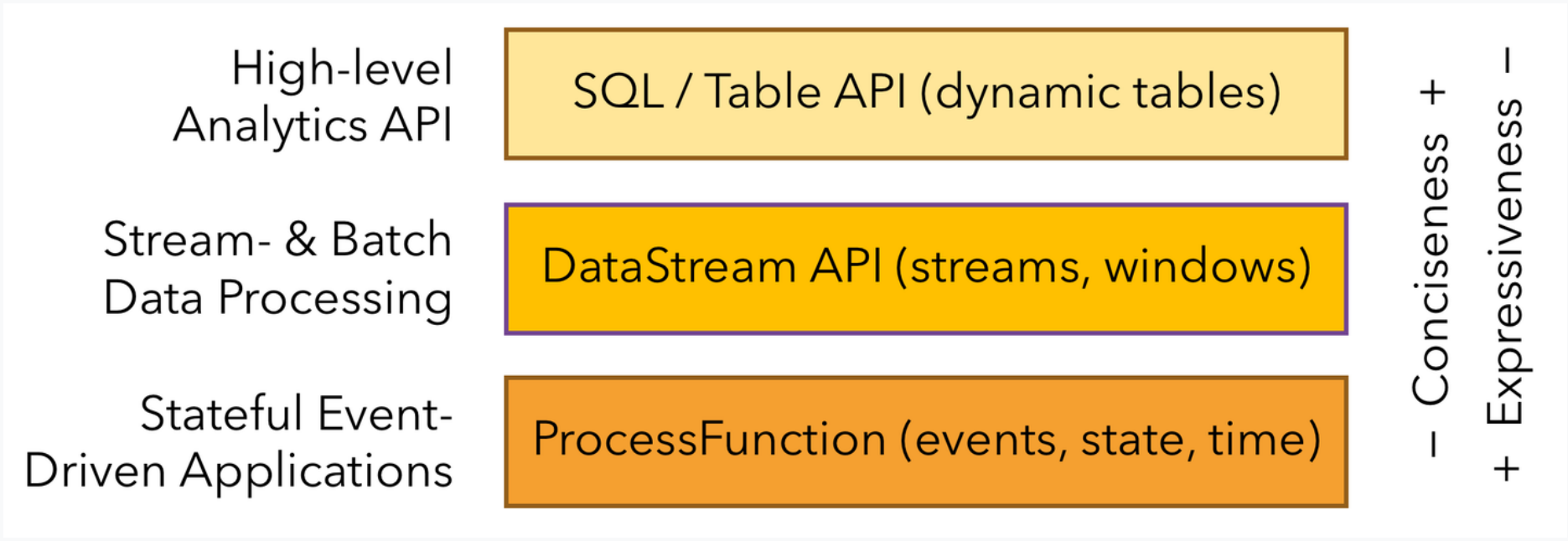
Leader election
Service discovery
State persistence

High availability service implementations:

APACHE ZooKeeper™



Flink Apis


High-level
Analytics API
SQL / Table API (dynamic tables)
Stream- & Batch
Data Processing
DataStream API (streams, windows)
Stateful Event-
Driven Applications
ProcessFunction (events, state, time)
– Conciseness +
+ Expressiveness –
SQL

Data stream API

```java
// a stream of website clicks
DataStream<Click> clicks = ...

DataStream<Tuple2<String, Long>> result = clicks
  // project clicks to userId and add a 1 for counting
  .map(
    // define function by implementing the MapFunction interface.
    new MapFunction<Click, Tuple2<String, Long>>() {
      @Override
      public Tuple2<String, Long> map(Click click) {
        return Tuple2.of(click.userId, 1L);
      }
    })
  // key by userId (field 0)
  .keyBy(0)
  // define session window with 30 minute gap
  .window(EventTimeSessionWindows.withGap(Time.minutes(30L)))
  // count clicks per session. Define function as lambda function.
  .reduce((a, b) -> Tuple2.of(a.f0, a.f1 + b.f1));
```

Strteaming API

```java
DataStream<Tuple2<String, String>> stream = ...;

DataStream<Tuple2<String, Long>> result = stream
    .keyBy(value -> value.f0)
    .process(new CountWithTimeoutFunction());


public class CountWithTimestamp {
    ...
}


public class CountWithTimeoutFunction
        extends KeyedProcessFunction<Tuple, Tuple2<String, String>, Tuple2<String, Long>> {

    private ValueState<CountWithTimestamp> state;

    @Override
    public void open(OpenContext openContext) throws Exception {
        state = getRuntimeContext().getState(new ValueStateDescriptor<>("myState", CountWithTimestamp.class));
    }

    @Override
    public void processElement(
            Tuple2<String, String> value,
            Context ctx,
            Collector<Tuple2<String, Long>> out) throws Exception {

        CountWithTimestamp current = state.value();
        if (current == null) {
            current = new CountWithTimestamp();
            current.key = value.f0;
        }

        current.count++;
        current.lastModified = ctx.timestamp();
        state.update(current);
        ctx.timerService().registerEventTimeTimer(current.lastModified + 60000);
    }

    @Override
    public void onTimer(
            long timestamp,
            OnTimerContext ctx,
            Collector<Tuple2<String, Long>> out) throws Exception {
        CountWithTimestamp result = state.value();
        if (timestamp == result.lastModified + 60000) {
            out.collect(new Tuple2<String, Long>(result.key, result.count));
        }
    }
}
```

Table API

```
Table result =
    table
        .window(
            Session
                .withGap(lit(30).minutes())
                .on($("clicktime"))
                .as("w"))
        .groupBy(
            $("w"),
            $("userId"))
        .select(
            $("userId"),
            $("userId").count());
```

SQL API

```
SELECT userId, COUNT(*)
FROM clicks
GROUP BY SESSION(clicktime, INTERVAL '30' MINUTE), userId
```